

NOOK

Property Rental Platform

Full Cycle Development Report

Institution ENSTA	Academic Year 2025–2026
Course Multidisciplinary Projects	Date April 2026

Project Team — PFP-CP2

Name	Role	Specialization
Namane Haithem	Team Lead / Backend	Backend & Project Mgmt
Akli Merouane	Backend Dev	Auth & API Integration
Baroudi Laid	Backend Dev	Models & Post Logic
Messikh Yahia	Backend Dev	Main Page Logic
Tetbirt Abdellah	Frontend Lead	Frontend & Auth UI
Djabed Alae dine	Frontend Dev	Pages & API Integration
Bennacer Sami Fares	UI/UX Designer	Design & Supervision
Hamed Mehdi Abdelouahab	Frontend Dev / Data	UI & Database Population

1. Abstract

Nook is a full-stack web application developed by an eight-member multidisciplinary team as a final project for the Multidisciplinary Projects course at ENSTA (2025–2026). The platform addresses the challenge of finding and managing property rentals in Algeria by providing a seamless digital interface for both tenants and hosts.

The application was built using a decoupled architecture: a Django (Python) backend exposing a RESTful API via Django Ninja, and a Next.js (React 19) frontend. Core features include secure JWT authentication, dynamic property listings with image uploads stored on Supabase (S3-compatible), an interactive Leaflet-based map, advanced search and filtering, and a complete reservation workflow with host approval capabilities.

The team adopted an Agile-inspired iterative methodology, holding weekly meetings and using GitHub for collaborative version control. Development spanned approximately ten weeks, from initial concept in February 2026 to final delivery in April 2026. This report documents the complete development lifecycle, from requirements gathering to final delivery and edge-case resolution.

2. Context & Problem Statement

2.1 Background

Finding rental accommodation in Algeria — particularly for students and professionals relocating to a new city — is a fragmented process. Property listings are scattered across social media groups, word of mouth, and informal networks, making it difficult for both tenants and landlords to manage bookings efficiently.

The PFP-CP2 team identified this gap and proposed Nook: a centralized digital platform that brings rental listings, property information, interactive maps, and booking management into a single, user-friendly web application.

2.2 Target Users

- **Tenants:** individuals seeking to browse, filter, and book rental properties.
- **Hosts / Landlords:** property owners who want to list, manage, and monetize their properties.
- **Administrators:** platform maintainers who oversee content moderation and user management.

2.3 Key Challenges

- Short development timeline (~10 weeks) with a large, inexperienced team.
- Experience gap between frontend and backend sub-teams, risking desynchronization.
- Integrating multiple external services (Supabase PostgreSQL, Supabase Storage, Redis).
- Ensuring consistent API contracts between a decoupled frontend and backend.

3. Objectives

The following SMART objectives were defined at project kickoff (04/02/2026):

Objective	Description	Status
Authentication	Implement secure JWT-based user registration and login with full error handling.	Completed
Property Management	Allow hosts to create, modify, and delete property listings with media uploads.	Completed
Search & Filter	Enable tenants to search properties by location, price range, and availability dates.	Completed
Reservations	Build a complete booking workflow: request, approval/rejection, and status tracking.	Completed
Interactive Maps	Display property locations using Leaflet maps integrated with the frontend.	Completed
Responsive UI	Design and implement a mobile-responsive UI matching the approved Figma mockups.	Completed
User Profiles	Allow users to view and modify their profile information and review history.	Completed

4. Functional Specifications

4.1 User-Facing Features

- **Browse & Discover:** Users can scroll through property listings on the home page, view photos, pricing, and location.
- **Advanced Search:** Filter by city/wilaya, price range, and availability dates via GET /api/Search/.
- **Property Detail View:** Full details page with image gallery, amenities list, host info, and integrated Leaflet map.
- **Secure Authentication:** Registration and login with JWT access/refresh tokens; persistent sessions via secure local storage.
- **Reservations:** Tenants select dates on a property page and submit a booking request; hosts approve or reject.
- **User Profile (MyNook):** Hosts view and manage their listings; tenants track upcoming and past reservations.
- **Profile Editing:** Users can update email, address, phone number, and other personal data.

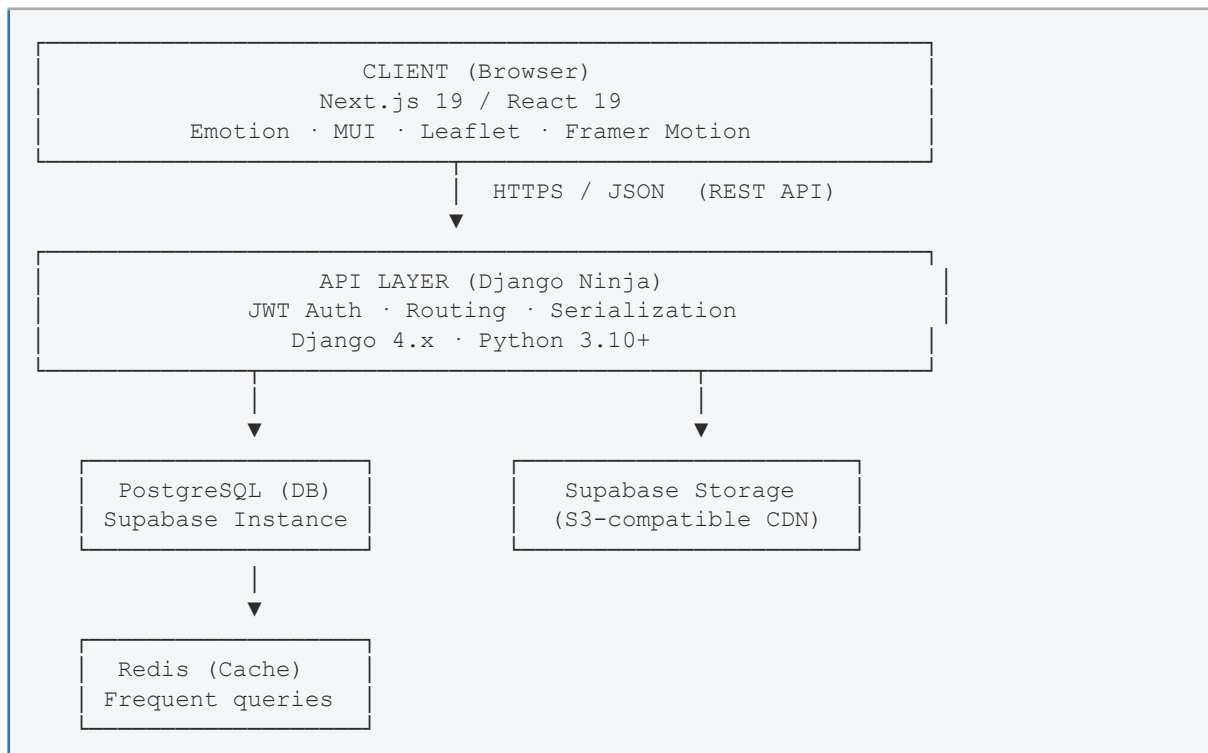
4.2 Host / Admin Features

- **Property Creation:** Multi-step form to add a property (Nook): title, description, price, location, photos.
- **Post Management:** Create posts linking a Nook to an availability period and price; manage active listings.
- **Media Uploads:** Images are uploaded directly to Supabase S3 storage via the backend.
- **Reservation Management:** Hosts can approve or reject pending booking requests from their dashboard.
- **Comments:** Users can post comments on property listings; host can respond.

5. System Architecture

5.1 Overview

Nook follows a decoupled three-tier architecture separating the presentation layer (Next.js), the logic layer (Django + Django Ninja), and the data layer (PostgreSQL on Supabase).



5.2 Backend Module Structure

- **Accounts** — User registration, login, profile management, and JWT issuance.
- **Houses** — Property (Nook) model, creation, modification, and retrieval endpoints.
- **Posts** — Listing posts linking a Nook to availability; main browsing feed endpoint.
- **Reservations** — Booking logic: creation, approval, rejection, and status tracking.
- **wilayas** — Location/region reference data for Algerian provinces.
- **config** — Django settings, ASGI/WSGI configuration, and global API router.
- **utilitymethods** — Shared helper functions and storage utilities.

5.3 Communication & Security

- **Authentication Flow:** On login, the backend issues a JWT access token (short-lived) and a refresh token. The frontend stores these tokens and attaches the access token in the Authorization: Bearer <token> header for every protected API request.
- **Data Format:** All API communication uses JSON over HTTP/HTTPS.
- **Media Storage:** Property images are sent from the frontend to the Django backend, which proxies them to Supabase S3 using Boto3. Public URLs are then stored in the database and served via CDN.
- **Caching:** Redis is used as a cache layer for frequently requested data (e.g., main property feed) to reduce database load.

6. Technology Stack

Layer	Technology	Version	Purpose
Frontend	Next.js / React	15 / 19	SPA framework, routing, SSR
Frontend	Material-UI (MUI)	v5	UI component library
Frontend	Emotion	v11	CSS-in-JS styling
Frontend	React Hook Form + Zod	latest	Form handling & validation
Frontend	Leaflet / React-Leaflet	v4	Interactive property maps
Frontend	Framer Motion	v11	UI animations
Frontend	Axios	v1	HTTP client for API calls
Backend	Django	v4.2	Web framework (Python)
Backend	Django Ninja	v1	REST API layer + Swagger
Backend	Django Ninja JWT	latest	JWT authentication
Backend	psycopg2	v2.9	PostgreSQL adapter
Backend	django-redis	v5	Redis caching integration
Backend	Boto3 / django-storages	latest	S3 media file storage
Database	PostgreSQL	v15	Relational database (Supabase)
Cache	Redis	v7	In-memory caching layer
Storage	Supabase Storage	—	S3-compatible file storage CDN

7. Methodology

7.1 Agile-Inspired Iterative Approach

The team adopted an Agile-inspired iterative workflow adapted to the academic context. Rather than formal Scrum sprints, the team organized work into weekly development cycles punctuated by regular meetings. This approach was chosen because it accommodates varying experience levels across team members while maintaining a clear pace of delivery.

7.2 Meeting Cadence

- **Weekly or bi-weekly meetings:** held in person (Résidence Universitaire Dergana) or virtually (Google Meet / Discord), chaired by Namane Haithem.
- **Minutes documented:** each meeting produced formal minutes recording agenda, progress, task assignments, decisions, and next steps.

- **Ad-hoc technical sessions:** conducted when urgent issues arose (e.g., authentication bugs, merge conflicts).

7.3 Version Control Strategy

- **Platform:** GitHub repository with a shared codebase for both frontend and backend.
- **Branching:** feature branches with pull requests (PRs) for major changes to avoid direct commits to main.
- **Communication protocol:** team members were required to notify the group before any significant modification to the codebase — a rule formalized after a coordination issue in the second week.

7.4 Team Organization

The team was divided into two sub-teams from the first meeting:

Backend Team	Frontend Team
• Baroudi Laid • Akli Merouane • Namane Haithem • Messikh Yahia	• Tetbirt Abdellah (Lead) • Djabed Alae dine • Bennacer Sami Fares (UX) • Hamed Mehdi Abdelouahab

8. Task Distribution

Team Member	Primary Responsibilities	Key Deliverables
Namane Haithem	Team Lead, Backend, Project Mgmt	DB schema, reservation endpoints, account/profile API, meeting minutes, project supervision
Akli Merouane	Backend Dev	Django models, DB migrations (Supabase), authentication bug fixes, API integration review
Baroudi Laid	Backend Dev	Comments model, post detail API, Nook creation & modification endpoints, hot fixes
Messikh Yahia	Backend Dev	Main page API endpoint, database logic for property feed
Tetbirt Abdellah	Frontend Lead	Login/signup UI, animations, auth page fixes, post page layout, mobile navbar
Djabed Alae dine	Frontend Dev	Main page frontend, MyNook & reservation page, Nook creation API integration
Bennacer Sami Fares	UI/UX Designer	Figma designs (all pages + mobile version), frontend quality supervision

Hamed Mehdi Abdelouahab	Frontend Dev / Data	Frontend pages, database population with sample data
----------------------------	---------------------	---

9. Development Timeline

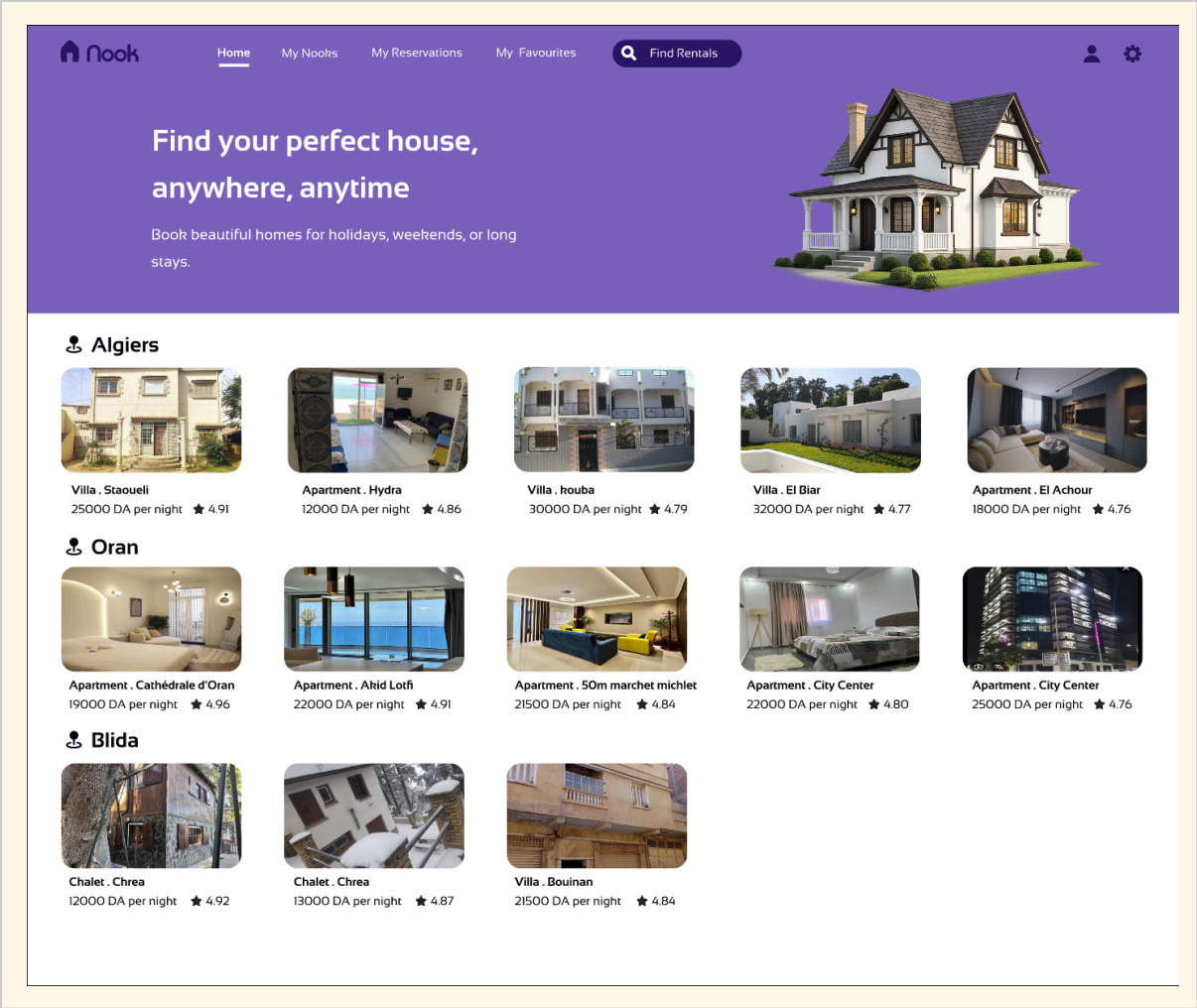
Period	Phase	Key Activities	Deliverable
Week 1–2	Inception	Project naming (Nook), team formation, role assignment, DB schema design, tech stack decision	Project charter, DB schema
Week 3–4	Foundation	Figma MVP design (all pages), login page coding, Django models & Supabase migration, auth backend	Figma mockups, working auth
Week 5–6	Core Dev	Main page, post page frontend; main page & post page backend; reservation logic	MVP frontend + API
Week 7–8	Feature Completion	Mobile-responsive version, account/profile page, MyNook, comments, post creation	Full feature set
Week 9	Integration & Fix	API integration fixes, model mismatches resolved, frontend-backend alignment, data population	Integrated application
Week 10	Finalization	Edge-case resolution, performance testing, documentation, final review & delivery	Production-ready release

10. Application Overview

The Nook application comprises the following main screens. Annotated screenshots should be inserted in the placeholders below using the actual application captures.

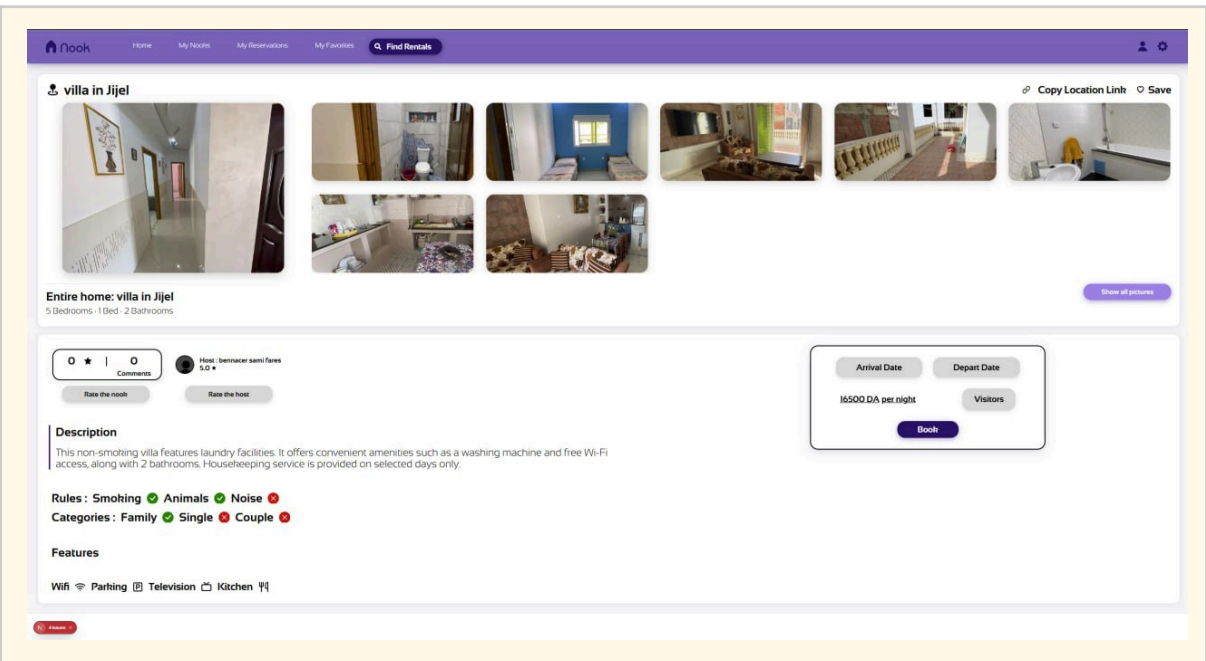
10.1 Home / Property Feed Page

Displays a paginated grid of property listings. Each card shows the property image, title, location (wilaya), price per night, and a quick link to the detail page. The search bar at the top allows filtering by location and dates.



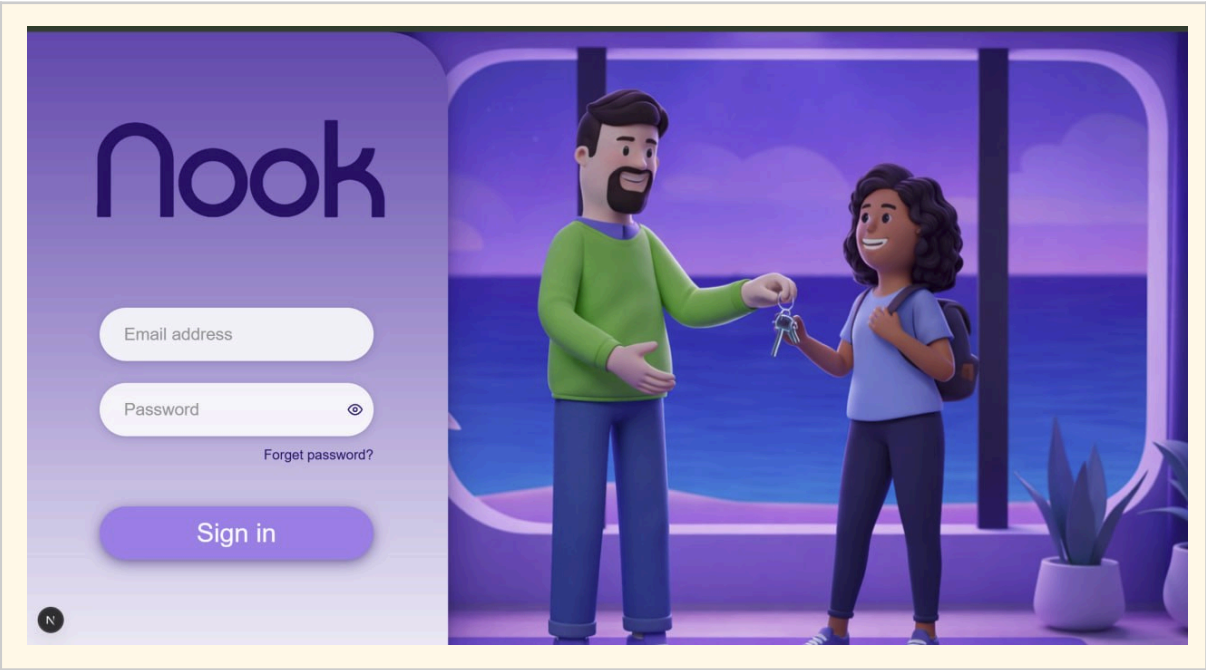
10.2 Property Detail Page

Shows the full property information: image gallery, description, host details, amenities, pricing, availability calendar, and an embedded Leaflet map displaying the exact location. A booking button initiates the reservation flow.



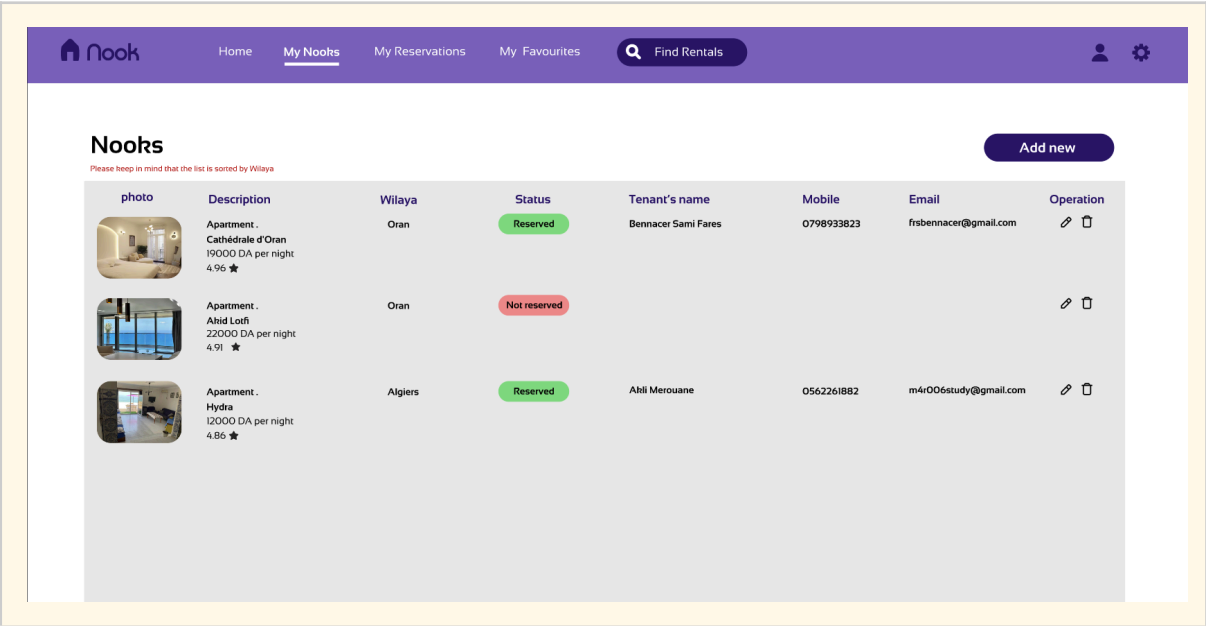
10.3 Authentication Pages

Registration and login forms built with React Hook Form and Zod validation. On successful login, JWT tokens are stored and the user is redirected to the home page. Comprehensive error messages guide users through input mistakes.



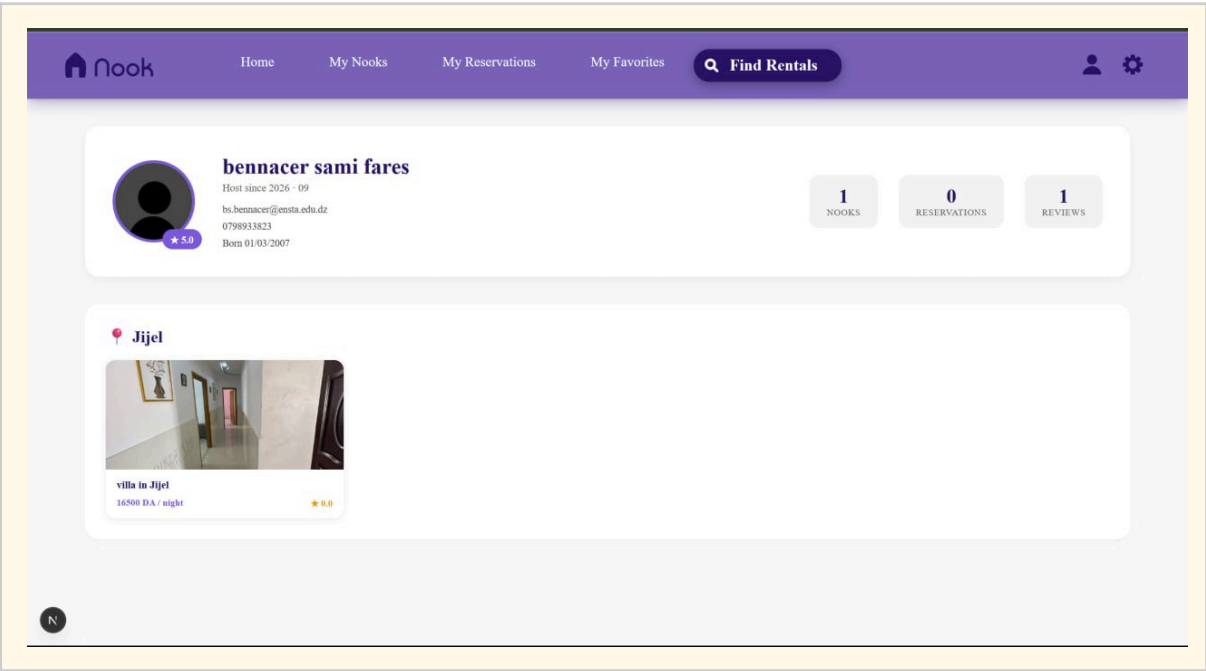
10.4 MyNook Dashboard

Hosts see their listed properties with management options (edit, delete, view reservations). Tenants see their upcoming and past reservations with status badges (Pending, Approved, Rejected).



10.5 User Profile Page

Displays user information (name, email, phone, address) with an edit mode for modifying personal details. Hosts also see aggregate statistics: number of active listings and total reservations received.



11. Testing & Validation

11.1 Manual Testing

Given the project timeline, the primary testing strategy was structured manual testing. Each developer was responsible for testing their own endpoints and UI components before merging. Integration testing was performed collectively during the weekly meetings.

Test Case	Method	Result	Notes
User registration with valid data	Manual (UI)	Pass	JWT tokens returned
User registration with duplicate email	Manual (UI)	Pass	Proper error message displayed
Login with wrong password	Manual (UI)	Pass	401 error handled gracefully
Create a property listing (host)	Manual (UI + API)	Pass	Images uploaded to Supabase S3
Search by wilaya & price range	Manual (UI)	Pass	Filtered results returned correctly
Submit reservation request	Manual (UI)	Pass	Reservation created with Pending status
Host approves reservation	Manual (API)	Pass	Status updated to Approved
Modify user profile data	Manual (UI)	Pass	Data persisted in database

Property map displays correctly	Manual (UI)	Pass	Leaflet renders pin at correct coordinates
Mobile responsive layout	Manual (Browser DevTools)	Pass	All pages adapt to 375px viewport

11.2 API Testing

All API endpoints were tested using the Django Ninja auto-generated Swagger UI (<http://127.0.0.1:8000/api/docs>) and curl commands. This allowed rapid iteration on request/response shapes without requiring the frontend to be running.

11.3 Known Limitations

- No automated test suite (unit or integration tests) was implemented due to time constraints — identified as a future improvement.
- Load testing was not conducted; performance under concurrent users remains untested.

12. Project Review & Challenges

12.1 Main Challenges Encountered

Communication & Coordination (Week 2)

Problem: Lack of communication between all members led to misunderstandings and merge conflicts when pushing to the repository.

Resolution: A dedicated meeting was held to enforce a communication protocol requiring team members to notify the group before any significant modification, and to prefer pull requests over direct commits.

Experience Gap Between Sub-Teams (Week 1–4)

Problem: The frontend sub-team had significantly more experience, risking desynchronization with the backend team.

Resolution: The backend team doubled their effort to learn Django Ninja in the first two weeks. The team also adopted a clear API contract early on (defined data shapes), decoupling frontend development from backend readiness.

API Integration Mismatches (Week 9)

Problem: Several API endpoints had response shapes that did not match what the frontend expected (especially account, MyNook, and reservation pages), causing runtime errors.

Resolution: A full integration session was held; Akli Merouane led a systematic review of all endpoints, and Namane Haithem updated the profile and account endpoints to return the correct data shapes.

Code Merging Conflicts (Week 7–9)

Problem: Direct commits to the main branch by multiple developers caused merge conflicts and broke the frontend build multiple times.

Resolution: The team formally adopted a pull-request-first policy for all changes touching shared modules.

13. Finalization & Edge Case Resolution

The final development phase (Week 10) was dedicated to hardening the application by identifying and resolving edge cases, improving robustness, and polishing the user experience before final delivery.

13.1 Authentication Edge Cases

- **Expired JWT token:** Implemented automatic token refresh logic in the frontend Axios interceptor. When a 401 response is received, the frontend silently requests a new access token using the refresh token before retrying the original request.
- **Invalid refresh token:** If the refresh token has also expired, the user is cleanly logged out and redirected to the login page with an informative message.
- **Duplicate email registration:** Added a database-level unique constraint on the email field and a user-friendly error message in the registration form.

13.2 Property Listing Edge Cases

- **Empty property feed:** Handled the case where no properties match the search criteria by displaying a descriptive empty-state component instead of an empty grid.
- **Image upload failures:** Added try/catch handling around S3 upload calls with user-facing error feedback if the upload fails. The property creation form is blocked from submission if required images fail to upload.
- **Invalid coordinates:** Added server-side validation to ensure latitude/longitude values are within Algeria's geographic bounding box before saving.

13.3 Reservation Edge Cases

- **Double booking:** Added a backend check to reject a new reservation request if the target property already has an approved reservation overlapping the requested dates.
- **Reservation on own property:** Added a guard preventing hosts from creating a reservation on their own listing.
- **Status transition integrity:** Restricted status changes so that only the host can approve/reject, and only while the reservation is in Pending state — preventing erroneous state transitions.

13.4 Frontend Robustness

- **Form validation:** All forms use Zod schemas to validate input client-side before submission, preventing unnecessary API calls.
- **API error display:** A global error boundary and per-request error states ensure that API failures are surfaced to the user rather than silently failing.
- **Loading states:** All data-fetching operations display skeleton loaders, preventing empty flash content on slow connections.
- **Mobile navigation:** The mobile version of the navigation bar was finalized to be fully functional across iOS Safari and Android Chrome.

13.5 Final Performance & Security Checks

- **Environment variable audit:** Removed all hardcoded credentials from the codebase and confirmed all sensitive values are loaded exclusively from .env files.
- **Unused dependency removal:** Cleaned the requirements.txt and package.json of unused packages to reduce bundle size and attack surface.
- **CORS configuration:** Verified that CORS settings in Django only allow the production frontend origin, not wildcards.

14. Future Improvements

- **Automated Testing:** Implement a pytest-based backend test suite and Jest/Cypress end-to-end tests for the frontend to ensure regression safety in future development.
- **Containerization:** Package the full stack using Docker and docker-compose to simplify onboarding and enable consistent deployment across environments.
- **Payment Integration:** Integrate an online payment gateway (e.g., Chargily for the Algerian market) to enable in-app rental payments.
- **Review & Rating System:** Extend the existing comments feature into a full 5-star review system with average scores displayed on listings.
- **Real-time Notifications:** Use WebSockets (Django Channels) to push real-time reservation status updates to hosts and tenants.
- **Admin Dashboard:** Build a dedicated admin panel for content moderation, user management, and analytics.
- **CI/CD Pipeline:** Set up GitHub Actions for automated linting, testing, and deployment on every pull request to main.

15. Conclusion

Nook is the result of ten weeks of collaborative work by an eight-member multidisciplinary team at ENSTA. Starting from a problem statement — the difficulty of finding and managing rental accommodation in Algeria — the team designed, built, and delivered a full-stack web application that addresses this need with a modern, responsive, and user-friendly interface.

The project provided the team with invaluable hands-on experience in full-stack development, REST API design, database modeling, cloud storage integration, and Agile team collaboration. Key lessons learned include the critical importance of communication protocols in distributed teams, the value of defining API contracts early to decouple parallel workstreams, and the need to allocate dedicated time for integration, testing, and edge-case resolution in any realistic project timeline.

While the current version of Nook achieves all core objectives defined at kickoff, it lays a solid foundation for the future improvements outlined in Section 14. The codebase is well-structured, documented, and maintainable, making it a viable starting point for continued development beyond this academic context.

16. Bibliography & References

- **Django Documentation:** <https://docs.djangoproject.com/en/4.2/>
- **Django Ninja:** <https://django-ninja.dev/>

- **Next.js Documentation:** <https://nextjs.org/docs>
- **React 19 Documentation:** <https://react.dev/>
- **Material-UI (MUI):** <https://mui.com/>
- **Leaflet.js:** <https://leafletjs.com/>
- **Supabase Documentation:** <https://supabase.com/docs>
- **Boto3 Documentation:**
<https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>
- **React Hook Form:** <https://react-hook-form.com/>
- **Zod Schema Validation:** <https://zod.dev/>
- **OWASP JWT Security Cheat Sheet:**
https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

17. Annexes

Annex A — Core API Endpoints

Method	Endpoint	Auth	Description
POST	/api/token/	No	Obtain JWT access & refresh tokens
POST	/api/token/verify/	No	Verify token validity
GET/POST	/api/Account/	Yes	Get / create user account
GET/PATCH	/api/Account/{id}/	Yes	View / update user profile
GET/POST	/api/Houses/	Yes	List / create property (Nook)
GET/PUT/DELETE	/api/Houses/{id}/	Yes	Manage specific Nook
GET/POST	/api/Posts/	No/Yes	Browse listings / create post
GET/POST	/api/Reservations/	Yes	List / create reservations
PATCH	/api/Reservations/{id}/	Yes	Approve or reject reservation
GET	/api/Search/	No	Search properties with filters

Annex B — Database Entity Summary

- **User:** id, email, username, password_hash, phone, address, role (tenant/host), created_at
- **House (Nook):** id, owner_id (FK User), title, description, location, wilaya, latitude, longitude, images[]
- **Post:** id, house_id (FK House), price_per_night, available_from, available_to, is_active
- **Reservation:** id, post_id (FK Post), tenant_id (FK User), check_in, check_out, status (Pending/Approved/Rejected), created_at
- **Comment:** id, post_id (FK Post), author_id (FK User), content, created_at
- **Wilaya:** id, name, code (Algerian province reference data)

Annex C — Project Folder Structure

```
PFP-CP2/
├── backEnd/
│   ├── Accounts/           # User management & auth
│   ├── config/             # Settings, routing
│   ├── Houses/             # Nook models & endpoints
│   ├── Posts/              # Listing endpoints
│   ├── Reservations/       # Booking logic
│   ├── utilitymethods/     # Shared helpers
│   ├── wilayas/            # Location data
│   ├── manage.py
│   └── requirements.txt
└── frontEnd/
    ├── pfp_front/
    │   ├── app/             # Next.js App Router pages
    │   ├── components/     # Reusable React components
    │   ├── lib/             # API client utilities
    │   ├── types/           # TypeScript type definitions
    │   ├── styles/          # Global CSS
    │   └── public/          # Static assets
```